Teste de Software

Refatoração de Testes e Detecção de Test Smells

Relatório do trabalho prático

Aluno(a): Arthur Candian
Matrícula: 804243

Projeto: test-smelly (UserService) — Jest + ESLint + eslint-plugin-jest

Outubro de 2026

1. Análise de Test Smells

A suíte original (`userService.smelly.test.js`) possui 5 testes: 4 passam e 1 está ignorado. Apesar de verde e de atingir 100% de cobertura, ela esconde problemas graves. Abaixo estão os três smells principais identificados manualmente e confirmados (ou não) pela ferramenta.

1.1 Conditional Test Logic (Lógica Condicional no Teste)

Onde: teste 2, “deve desativar usuários se eles não forem administradores” (um `for` com `if/else` envolvendo os `expect`). **Por que é um mau cheiro:** o teste passa a ter caminhos de execução diferentes, ficando tão complexo quanto o código que verifica; quem lê precisa “executar o loop de cabeça” para saber o que está sendo afirmado. **Risco:** asserções podem simplesmente não ser executadas (se a condição mudar, o ramo deixa de rodar sem aviso), e, quando falha, não fica claro para qual usuário (comum ou admin) o comportamento quebrou. **Detectado pelo ESLint:** sim (`jest/no-conditional-expect`, linhas 44, 46 e 49).

1.2 Fragile Test (Teste Frágil)

Onde: teste 3, “deve gerar um relatório de usuários formatado”, que monta a linha exata `` `ID: ${id}, Nome: Alice, Status: ativo\n` `` e exige que o relatório comece exatamente com o título. **Por que é um mau cheiro:** o teste está acoplado ao *formato* da string (espaços, vírgulas, quebra de linha, ordem dos campos), e não ao comportamento (o relatório contém os dados dos usuários). O próprio código-fonte avisa que “o formato do relatório pode mudar”. **Risco:** qualquer mudança cosmética quebra o teste sem existir bug (alarme falso), gerando custo de manutenção e descrédito na suíte. **Detectado pelo ESLint:** não — exige análise humana.

1.3 Eager Test / Assertion Roulette (Teste Ansioso)

Onde: teste 1, “deve criar e buscar um usuário corretamente”, que executa duas ações distintas (`createUser` e `getUserById`) e faz três asserções sem mensagem. **Por que é um mau cheiro:** o teste verifica mais de um comportamento; o nome com “e” já denuncia isso. **Risco:** quando falha, é preciso investigar qual das responsabilidades quebrou; a primeira asserção que falha interrompe o teste e esconde as demais. O mesmo ocorre no teste 2. **Detectado pelo ESLint:** não com as regras configuradas.

1.4 Outros smells encontrados

Smell	Onde	Problema	ESLint
Teste “falso verde” (assert no *catch*)	Teste 4 (menor de idade)	Se `createUser` **não** lançar erro, o `catch` nunca roda e o teste passa. A regra de negócio pode ser removida sem que nenhum teste falhe (comprovado na seção 2).	Sim: `no-conditional-expect` (L73)
Ignored Test	Teste 5 (`test.skip`)	Teste vazio e desativado (TODO): dá falsa sensação de que o cenário “nenhum usuário” está coberto.	Sim: `no-disabled-tests` e `expect-expect` (L77)
Nomes vagos / dado compartilhado	Fixture global e título do teste 2	Dados definidos longe do teste e nomes que não dizem o resultado esperado dificultam a leitura isolada.	Não

2. Processo de Refatoração

O teste mais problemático é o de desativação (teste 2): reúne lógica condicional, vários cenários e asserções que podem nunca rodar. Foi dividido em quatro testes pequenos, cada um com um único motivo para falhar.

Antes (`userService.smelly.test.js`)

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);
  const todosOsUsuarios = [usuarioComum, usuarioAdmin];

  for (const user of todosOsUsuarios) {                // <- loop
    const resultado = userService.deactivateUser(user.id);
    if (!user.isAdmin) {                               // <- condicional
      expect(resultado).toBe(true);
      const usuarioAtualizado = userService.getUserById(user.id);
      expect(usuarioAtualizado.status).toBe('inativo');
    } else {
      expect(resultado).toBe(false);
    }
  }
});
```

Depois (`userService.clean.test.js`)

```
describe('deactivateUser', () => {
  it('deve retornar true ao desativar um usuário comum', () => {
    // Arrange
    const usuarioComum = criarUsuario(userService, { isAdmin: false });
    // Act
    const resultado = userService.deactivateUser(usuarioComum.id);
    // Assert
    expect(resultado).toBe(true);
  });

  it('deve alterar o status do usuário comum para "inativo" após desativá-lo', () => {
    // Arrange
    const usuarioComum = criarUsuario(userService, { isAdmin: false });
    // Act
    userService.deactivateUser(usuarioComum.id);
    // Assert
    expect(userService.getUserById(usuarioComum.id).status).toBe('inativo');
  });

  it('deve retornar false ao tentar desativar um administrador', () => { /* AAA idêntico */ });
  it('deve manter o administrador "ativo" após tentativa de desativação', () => { /* AAA */ });
});
```

Decisões tomadas e smells corrigidos:

• **Conditional Test Logic:** removidos `for`/`if`. Cada caso (comum e admin) virou um teste explícito, onde todas as asserções sempre executam.
• **Eager Test / Assertion Roulette:** resultado retornado e efeito colateral (status) foram separados. Uma falha agora aponta exatamente o comportamento quebrado pelo próprio nome do teste.
• **Padrão AAA:** blocos Arrange/Act/Assert marcados em todos os testes; os dados relevantes ao cenário (como `isAdmin`) ficam visíveis dentro do teste, e um builder (`criarUsuario`) preenche apenas o que é irrelevante.
• **Nomes descritivos** no formato “deve <resultado> quando <condição>”, organizados em `describe` por método.
• **Cobertura de cenários ausentes** (admin permanece ativo, id inexistente, campos obrigatórios, limite de 18 anos), e o teste ignorado foi implementado, não apenas removido.

Teste 4: de “falso verde” a teste que realmente falha

```
// ANTES: passa mesmo sem validação
try {
  userService.createUser(
    'Menor', 'menor@email.com', 17);
} catch (e) {
  expect(e.message).toBe(
    'O usuário deve ser maior de idade.');
}
```

```
// DEPOIS: falha se a exceção não for lançada
const criarMenorDeIdade = () =>
  userService.createUser(
    'Menor', 'menor@email.com', 17);

expect(criarMenorDeIdade).toThrow(
  'O usuário deve ser maior de idade.');
```

Prova de eficácia (mutação manual): removi a validação `if (idade < 18)` do `UserService` (bug injetado). A suíte **original continuou 100% verde** (4 passed, 1 skipped) — o bug passaria despercebido. A suíte **refatorada falhou em 2 testes** (menor de idade), detectando o defeito. Isso resolve, na raiz, o problema do trabalho anterior: testes que passam sem proteger o código.

3. Relatório da Ferramenta (ESLint)

Configuração: `eslint 8.57.1 + eslint-plugin-jest 29.x`, com `.eslintrc.json` na raiz (extends *eslint:recommended* e *plugin:jest/recommended*, mais as regras `no-disabled-tests`, `no-conditional-expect` e `no-identical-title`). Observação: foi usada a versão 8 porque o formato `.eslintrc.json` foi removido como padrão do ESLint 9+ (que exige *flat config*).

Primeira execução — `npx eslint .` (suíte original):

```
$ npx eslint .

/home/.../test-smelly/test/userService.smelly.test.js
  44:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  46:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  49:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  73:7  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  77:3  warning  Tests should not be skipped            jest/no-disabled-tests
  77:3  warning  Test has no assertions                 jest/expect-expect

x 6 problems (4 errors, 2 warnings)
```

Validação final — `npx eslint test/userService.clean.test.js` (sem nenhuma saída = 0 erros, 0 avisos) e `npm test`:

```
$ npx eslint test/userService.clean.test.js
$                                  <- nenhum erro ou aviso

$ npm test
PASS test/userService.clean.test.js   (25 testes)
PASS test/userService.smelly.test.js  (4 passed, 1 skipped)
Test Suites: 2 passed, 2 total
Tests:       1 skipped, 29 passed, 30 total       Cobertura: 100% stmts / branch / funcs / lines
```

Como a ferramenta automatizou a detecção: o ESLint percorreu a árvore sintática de todos os arquivos e apontou, em segundos, as linhas exatas com *expect* condicional (smells 1.1 e do teste 4) e o teste ignorado/sem asserção, sem leitura humana. Porém, a análise estática tem limites: **não detectou** o Fragile Test nem o Eager Test, que dependem de semântica e intenção (o que é “comportamento” versus “formato”). Por isso as duas abordagens se complementam: a ferramenta cobre as regras mecânicas; a revisão humana cobre o design do teste. Também é possível endurecer a configuração com regras adicionais como `jest/prefer-expect-assertions` ou `jest/max-expects`, que ajudariam a flagrar os testes ansiosos.

4. Conclusão

Este trabalho mostrou que um teste verde não significa um teste confiável: a suíte original passava, tinha 100% de cobertura e, ainda assim, deixaria passar a remoção de uma regra de negócio. Testes limpos — pequenos, com um único comportamento, seguindo AAA, sem lógica condicional e com nomes que documentam o requisito — funcionam como especificação executável, falham pelo motivo certo e custam menos para manter, porque não quebram a cada mudança cosmética. A análise estática com ESLint adiciona uma barreira automática e barata, que pode rodar no CI e impedir que novos smells entrem no projeto, enquanto a revisão humana trata o que a ferramenta não alcança. Em conjunto, essas práticas aumentam a confiança nas alterações, reduzem a dívida técnica nos testes e sustentam a evolução do software no longo prazo.